HW 4.3/4.4: Reproducible Data Analysis

Kareenita Kumar

1. Busiest Airports Analysis

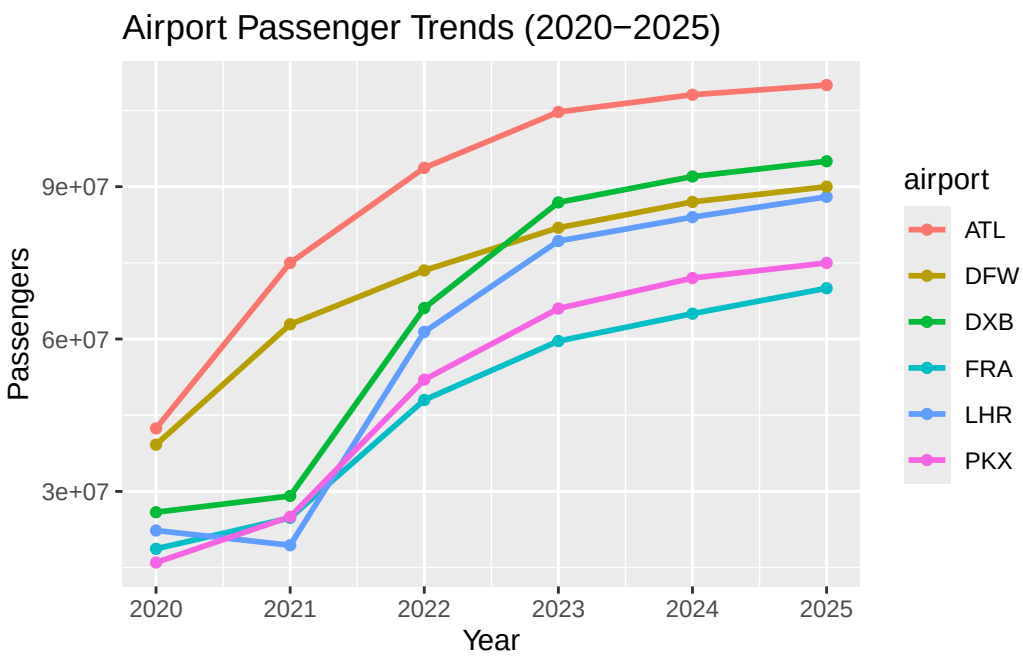
```r
airports_raw <- tibble(
  airport = c("ATL", "FRA", "PKX", "LHR", "DXB", "DFW"),
  `2020` = c(42400000, 18700000, 16000000, 22300000, 25900000, 39200000),
  `2021` = c(75000000, 24800000, 25000000, 19400000, 29100000, 62900000),
  `2022` = c(93700000, 48000000, 52000000, 61400000, 66100000, 73500000),
  `2023` = c(104700000, 59600000, 66000000, 79300000, 86900000, 81900000),
  `2024` = c(108100000, 65000000, 72000000, 84000000, 92000000, 87000000),
  `2025` = c(110000000, 70000000, 75000000, 88000000, 95000000, 90000000)
)
airports_tidy <- airports_raw %>%
  pivot_longer(
    cols = -airport,
    names_to = "year",
    values_to = "passengers"
  ) %>%
  mutate(year = as.numeric(year))
```

```r
library(knitr)
library(kableExtra)

airports_raw %>%
  kable() %>%
  kable_styling(full_width = FALSE)
```

airport	2020	2021	2022	2023	2024	2025
ATL	42400000	75000000	93700000	104700000	108100000	1.1e+08
FRA	18700000	24800000	48000000	59600000	65000000	7.0e+07
PKX	16000000	25000000	52000000	66000000	72000000	7.5e+07
LHR	22300000	19400000	61400000	79300000	84000000	8.8e+07

DXB	25900000	29100000	66100000	86900000	92000000	9.5e+07
DFW	39200000	62900000	73500000	81900000	87000000	9.0e+07

```
library(ggplot2)

ggplot(airports_tidy, aes(x = year, y = passengers, color = airport)) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(
    title = "Airport Passenger Trends (2020-2025)",
    x = "Year",
    y = "Passengers"
  )
```

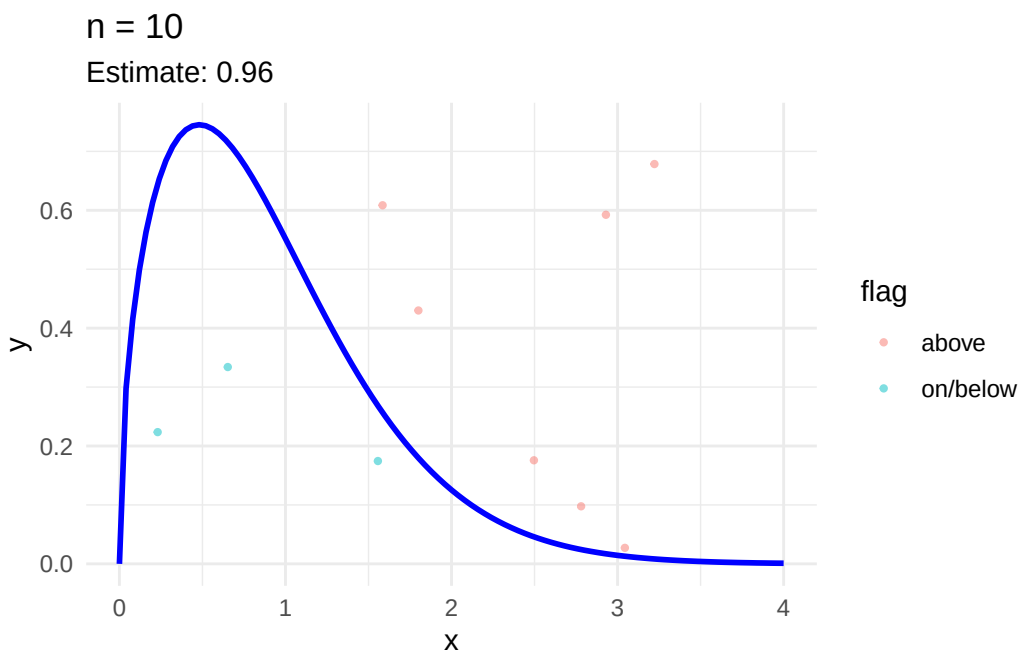


The data shows consistent passenger growth across all airports from 2020 to 2025. ATL and DFW consistently remain among the busiest airports, with strong recovery after 2020. Asian airports such as PKX show rapid growth, indicating increasing international travel demand in the region. Overall, all airports demonstrate a positive upward trend, with no sustained declines after 2020.

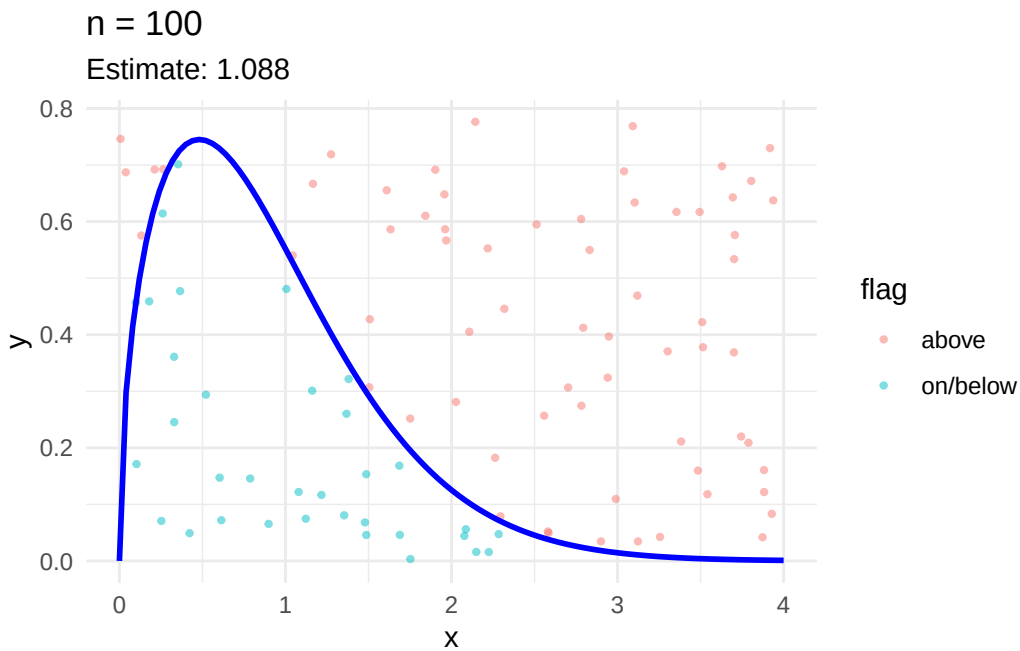
2. Monte Carlo Numerical Integration

```r
library(tidyverse)
library(ggplot2)

mc_simulation <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, x_min, x_max),
    y = runif(n, y_min, y_max)
  )
}

f <- function(x) dweibull(x, shape = 1.5, scale = 1)

x_min <- 0
x_max <- 4
y_min <- 0
y_max <- 0.8

rect_area <- (x_max - x_min) * (y_max - y_min)

make_plot <- function(n) {
  sim_data <- mc_simulation(n, x_min, x_max, y_min, y_max) %>%
    mutate(
      f_x = f(x),
      flag = ifelse(y <= f_x, "on/below", "above")
    )

  estimate <- mean(sim_data$flag == "on/below") * rect_area

  ggplot(sim_data, aes(x, y, color = flag)) +
    geom_point(alpha = 0.5, size = 0.8) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(x_min, x_max),
      linewidth = 1,
      color = "blue"
    ) +
    labs(
      title = paste("n =", n),
      subtitle = paste("Estimate:", round(estimate, 4)),
      x = "x",
      y = "y"
```

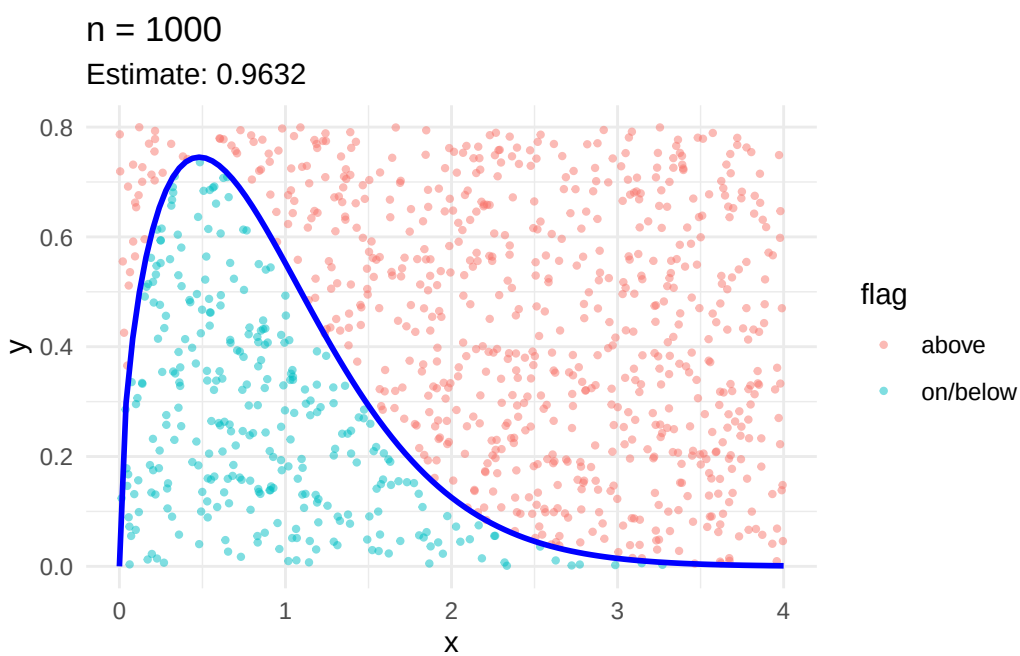
```r
    ) +
    theme_minimal()
}

# Store plots properly for Quarto
p1 <- make_plot(10)
p2 <- make_plot(100)
p3 <- make_plot(1000)
p4 <- make_plot(10000)

p1
```



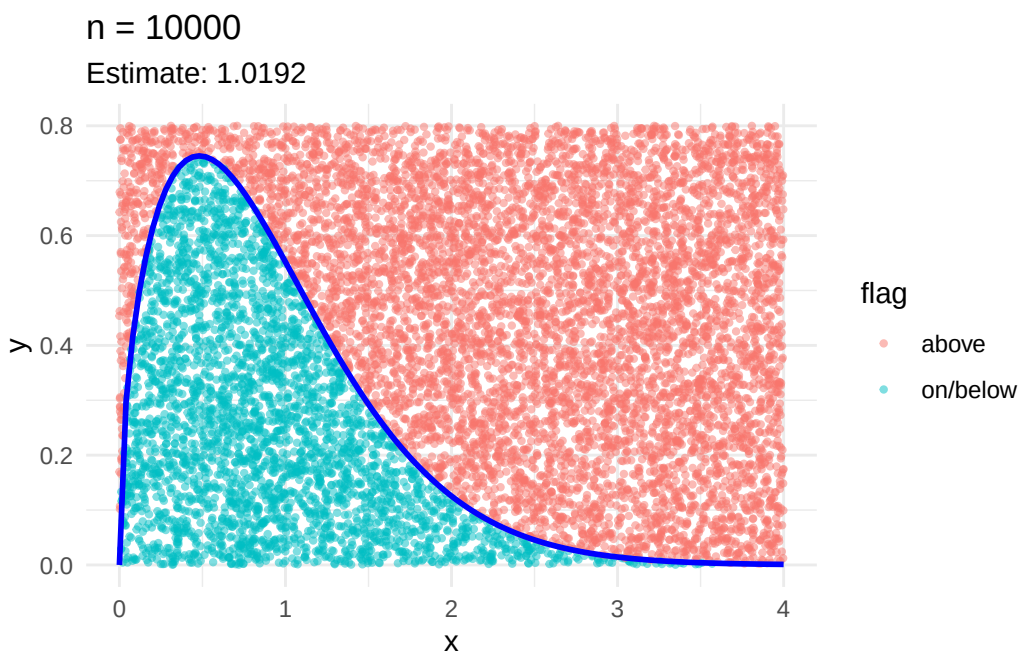

```r
p2
```


n = 100
Estimate: 1.088
0.8
0.6
0.4
0.2
0.0
y
0
1
2
3
4
x
flag
above
on/below

```
p3
```


n = 1000
Estimate: 0.9632
0.8
0.6
0.4
0.2
0.0
y
0
1
2
3
4
x
flag
above
on/below

```
p4
```



The Monte Carlo simulation approximates the integral of the Weibull probability density function by generating random points uniformly over a rectangular region that bounds the curve. Points are classified as either lying below or above the function and the proportion of points falling below the curve is multiplied by the area of the bounding rectangle to estimate the integral.

As the sample size n increases, the estimate becomes more stable and converges toward the true value of the integral. For small values of nnn, such as 10 or 100, the estimates exhibit high variability due to random sampling error. However, as nnn increases to 1,000 and 10,000, the simulated points more densely fill the region, reducing variance and improving accuracy. This demonstrates the Law of Large Numbers, where increasing the number of simulations leads to convergence of the estimator to the true value.

The small-multiple visualization clearly shows this convergence behavior, as the scatter of points becomes more uniform and the estimated area stabilizes across higher values of nnn. This illustrates the effectiveness of Monte Carlo methods for numerical integration, especially in cases where analytical solutions are difficult or unavailable.

3. Planning and Prompting GenAI Tools
4. Goal: Understand how calcium values change across time and compare patterns between the two repeated measurement groups in the data table.
5. Needs

nouns: data table, calcium measurements, time points, groups, rows, subjects, variables, tidy date, plot, trends

verbs: import, clean, rename, reshape, pivot, separate, group, visualize, compare, interpret

6. Steps

    1. Import data table into R
    2. Inspect the structure
    3. Identify any duplicates of columns
    4. Rename columns to make them more distinguishable
    5. Convert data from wide to long format using pivot_longer()
    6. Create variables for:
        - group (A vs B)
        - time
        - calcium value
    7. Remove missing values where needed
    8. Create visualization
        - line plot of calcium over time
        - color by group
    9. Interpret differences in trends between groups

```r
# Load libraries
library(tidyverse)


df <- read.csv("/Users/kareenitakumar/Downloads/calcium.csv")

# Preview data
head(df)
```

```
    y0   y1   y2   y3 y0.1 y1.1 y2.1 y3.1
1 87.3 86.9 86.7 75.5 83.8 85.5 86.2 81.2
2 59.0 60.2 60.0 53.6 65.3 66.9 67.0 60.6
3 76.7 76.5 75.7 69.5 81.2 79.5 84.5 75.2
4 70.6 76.1 72.1 65.3 75.4 76.7 74.3 66.7
5 54.9 55.1 57.2 49.0 55.3 58.3 59.1 54.2
6 78.2 75.3 69.1 67.6 70.3 72.3 70.6 68.6
```

```r
colnames(df)
```

```
[1] "y0"   "y1"   "y2"   "y3"   "y0.1" "y1.1" "y2.1" "y3.1"
```

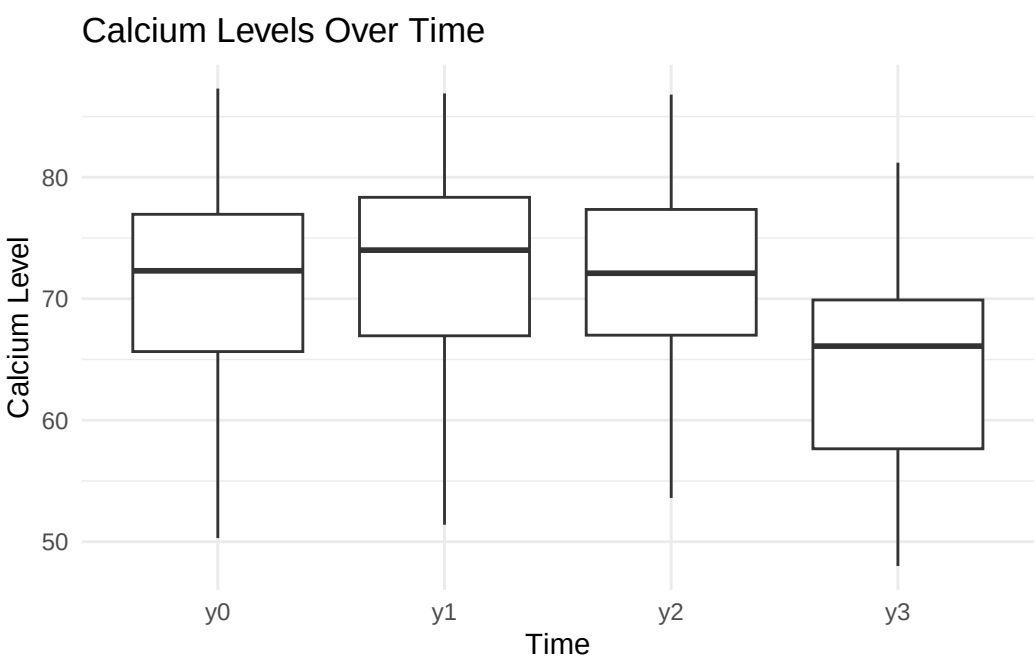
```r
left_cols <- df[, 1:4]
right_cols <- df[, 5:8]

colnames(left_cols) <- c("y0", "y1", "y2", "y3")
colnames(right_cols) <- c("y0", "y1", "y2", "y3")


df_combined <- bind_rows(left_cols, right_cols)

df_combined$Subject_ID <- seq_len(nrow(df_combined))


df_long <- df_combined %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "Time",
    values_to = "Calcium"
  )

# Order time correctly
df_long$Time <- factor(df_long$Time, levels = c("y0", "y1", "y2", "y3"))


# Summary statistics

summary_stats <- df_long %>%
  group_by(Time) %>%
  summarise(
    mean = mean(Calcium, na.rm = TRUE),
    sd   = sd(Calcium, na.rm = TRUE),
    min  = min(Calcium, na.rm = TRUE),
    max  = max(Calcium, na.rm = TRUE)
  )

print(summary_stats)
```

```
# A tibble: 4 x 5
```

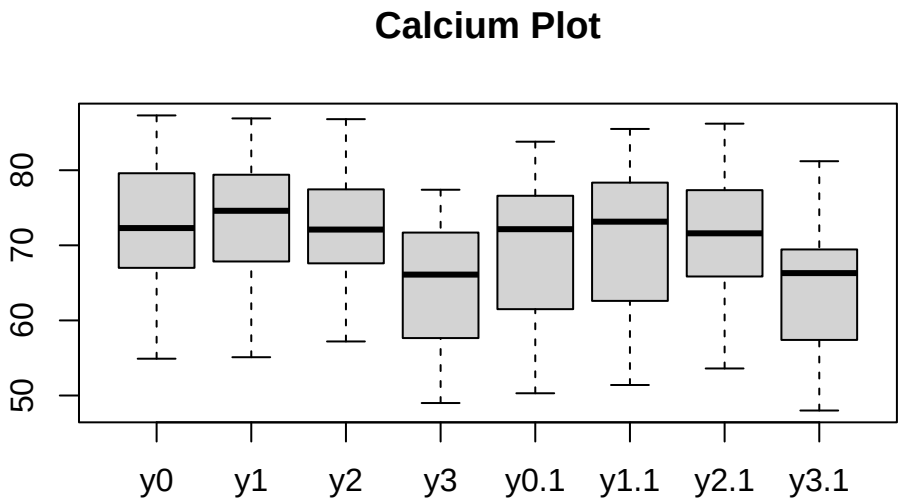
```
  Time   mean    sd   min   max
  <fct> <dbl> <dbl> <dbl> <dbl>
1 y0     70.8  9.72  50.3  87.3
2 y1     71.9  9.68  51.4  86.9
3 y2     71.8  8.78  53.6  86.8
4 y3     64.7  8.71  48    81.2
```

```r
# Visualization

ggplot(df_long, aes(x = Time, y = Calcium)) +
  geom_boxplot() +
  labs(
    title = "Calcium Levels Over Time",
    x = "Time",
    y = "Calcium Level"
  ) +
  theme_minimal()
```

```
Warning: Removed 4 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



```r
df <- read.csv("/Users/kareenitakumar/Downloads/calcium.csv")

boxplot(df, main="Calcium Plot")
```



Reflection:

In this course so far, I have learned how to approach data analysis in a more structured, reproducible, and intentional way. One of the biggest improvements in my work has been understanding how to move from simply producing outputs to building a full analysis that is clearly organized and easy for others to follow.

For example, in this assignment I learned the significance of separating data analysis into clear steps such as importing data, looking at structure, cleaning variables, reshaping data, and then visualizing results. Previously, I often focused only on generating a plot, but I now understand that the steps before visualization are just as important for ensuring the analysis is correct and meaningful. In the calcium data set specifically, I learned how converting data from wide to long format using pivot_longer() is important for creating a proper ggplot2 visual.

Another major learning experience was using Quarto to structure a reproducible document. I learned that keeping narrative text separate from code improve readability and helps communicate results easier.

I also learned how generative AI tools can support data analysis when used correctly. For instance, when I used a generic prompt like asking for a plot, the output was very limited and skipped important steps. However, when I used a plan-informed prompt that included specific

steps (tidying, reshaping, and summarizing data), the AI produced a much more complete workflow. This helped me understand that the quality of AI output depends heavily on how well the prompt is structured.

Overall, this course has helped me understand that data analysis is not just about coding, but about planning, structure, and communication. I now see reproducibility and clarity as essential parts of any analysis, not optional additions.

GenAI Usage Appendix

GenAI Usage 1:

Tool Used: ChatGPT

Prompt: create a plot of calcium.csv using R

Response:

df <- read.csv("calcium.csv")

boxplot(df, main="Calcium Plot")

GenAI Usage 2:

Tool Used: ChatGPT

Prompt: Develop a structured plan for tidying calcium.csv, reshaping it into long format, computing summary statistics, and creating a boxplot visualization in R using Quarto. Then generate full reproducible code.

Response:

```
library(tidyverse)

df <- read.csv("calcium.csv")

left_cols <- df[, 1:4]
right_cols <- df[, 5:8]

colnames(left_cols) <- c("y0","y1","y2","y3")
colnames(right_cols) <- c("y0","y1","y2","y3")

df_combined <- bind_rows(left_cols, right_cols)
df_combined$Subject_ID <- seq_len(nrow(df_combined))

df_long <- df_combined %>%
pivot_longer(
cols = starts_with("y"),
names_to = "Time",
```

values_to = “Calcium”
)

df_long$Time <- factor(df_long$Time,
levels = c(“y0”,“y1”,“y2”,“y3”))

summary_stats <- df_long %>%
group_by(Time) %>%
summarize(
mean = mean(Calcium, na.rm = TRUE),
sd = sd(Calcium, na.rm = TRUE),
min = min(Calcium, na.rm = TRUE),
max = max(Calcium, na.rm = TRUE)
)

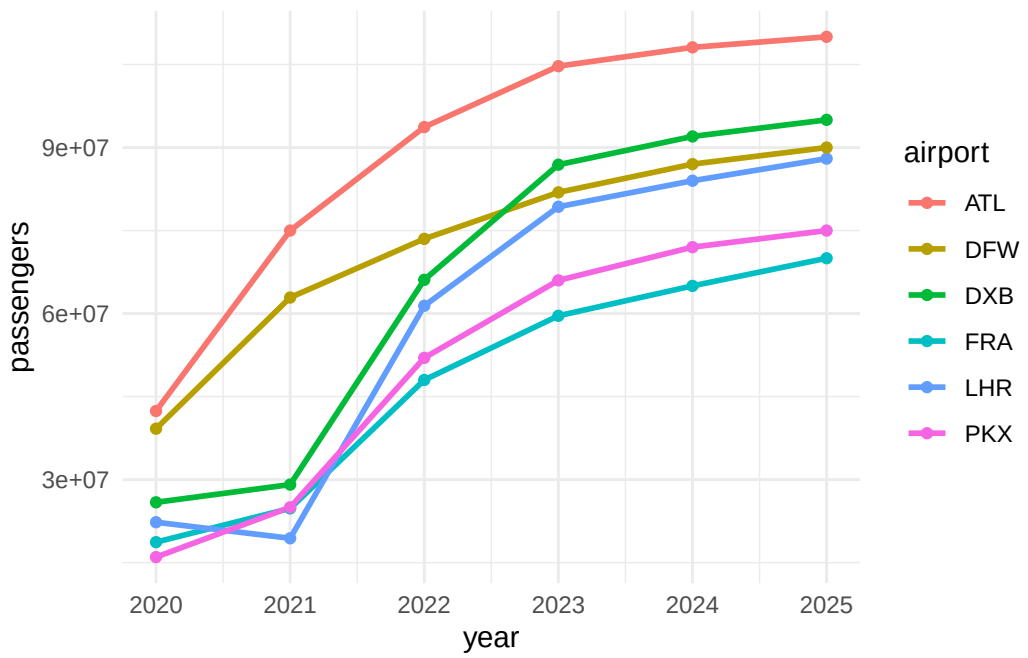
ggplot(df_long, aes(x = Time, y = Calcium)) +
geom_boxplot() +
theme_minimal()

Comparison: The generic prompt produced a very limited response that only generated a basic plot without any data preparation or structured workflow. In contrast, the plan-informed prompt produced a complete analysis pipeline including data cleaning, reshaping, summary statistics, and visualization. This demonstrates that structured prompting leads to significantly more complete and reproducible outputs.

Code Appendix

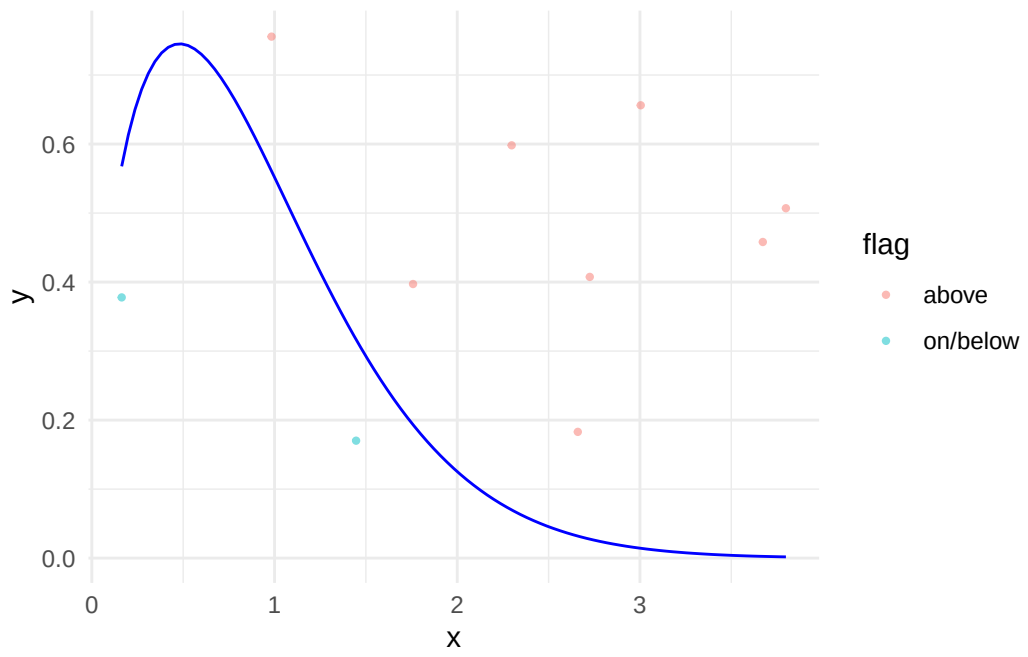
```
## Libraries

library(tidyverse)
library(ggplot2)

## Busiest Aiports Analysis
airports_raw <- tibble(
  airport = c("ATL", "FRA", "PKX", "LHR", "DXB", "DFW"),
  `2020` = c(42400000, 18700000, 16000000, 22300000, 25900000, 39200000),
  `2021` = c(75000000, 24800000, 25000000, 19400000, 29100000, 62900000),
  `2022` = c(93700000, 48000000, 52000000, 61400000, 66100000, 73500000),
  `2023` = c(104700000, 59600000, 66000000, 79300000, 86900000, 81900000),
  `2024` = c(108100000, 65000000, 72000000, 84000000, 92000000, 87000000),
  `2025` = c(110000000, 70000000, 75000000, 88000000, 95000000, 90000000)
)

airports_tidy <- airports_raw %>%
```

```r
  pivot_longer(
    cols = -airport,
    names_to = "year",
    values_to = "passengers"
  ) %>%
  mutate(year = as.numeric(year))

ggplot(airports_tidy, aes(x = year, y = passengers, color = airport)) +
  geom_line(linewidth = 1) +
  geom_point() +
  theme_minimal()
```

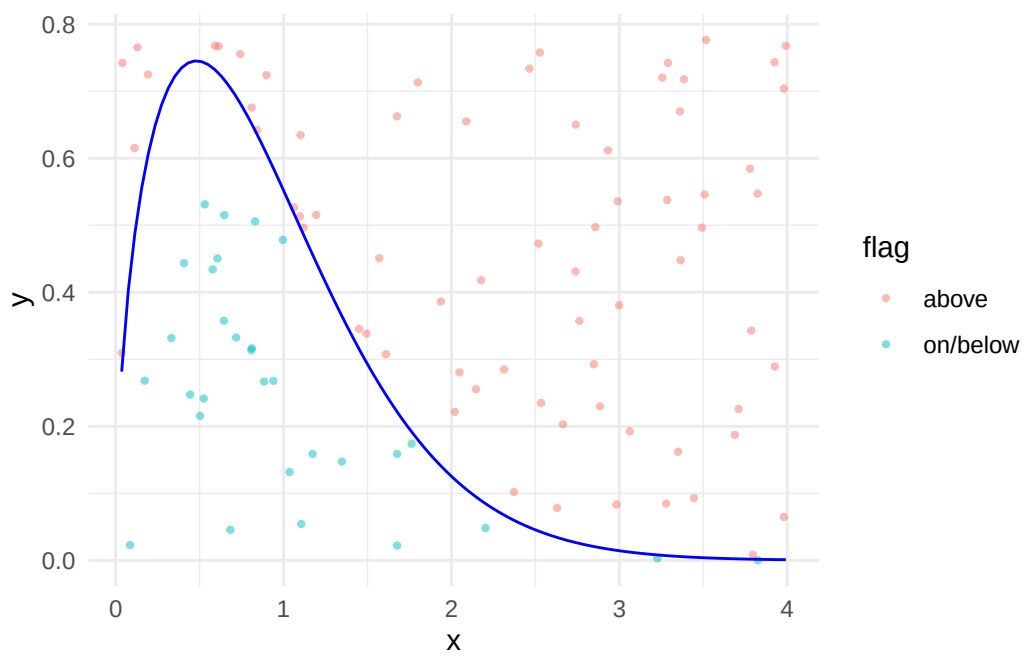



```r
## Monte Carlo Numerical Integration
mc_simulation <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, x_min, x_max),
    y = runif(n, y_min, y_max)
  )
}

f <- function(x) dweibull(x, shape = 1.5, scale = 1)
```

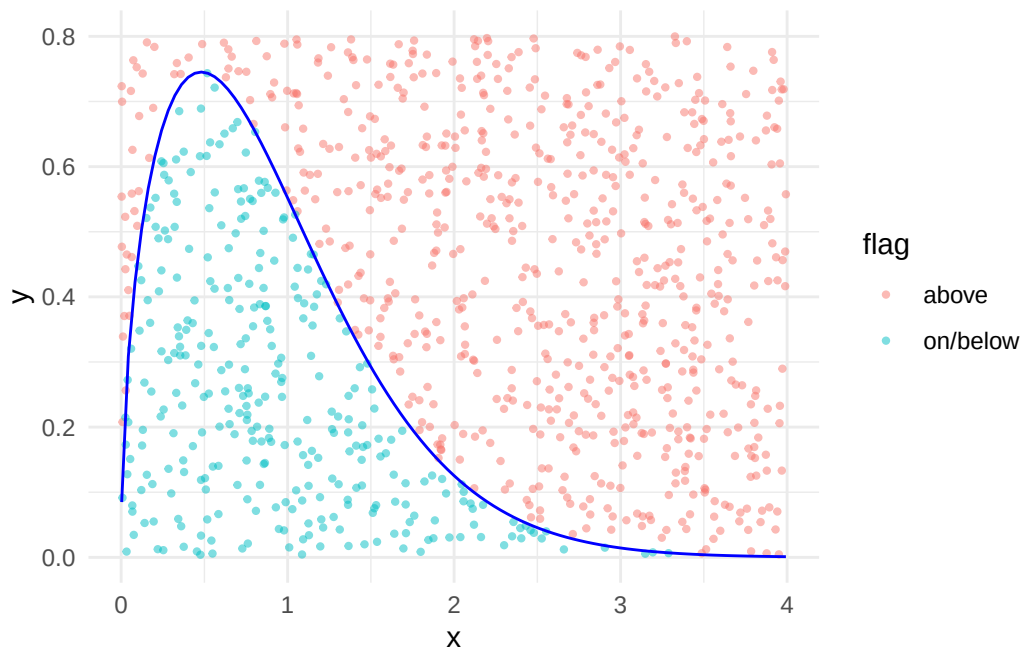
```r
x_min <- 0
x_max <- 4
y_min <- 0
y_max <- 0.8

rect_area <- (x_max - x_min) * (y_max - y_min)

make_plot <- function(n) {
  sim_data <- mc_simulation(n, x_min, x_max, y_min, y_max) %>%
    mutate(
      f_x = f(x),
      flag = ifelse(y <= f_x, "on/below", "above")
    )

  estimate <- mean(sim_data$flag == "on/below") * rect_area

  ggplot(sim_data, aes(x, y, color = flag)) +
    geom_point(alpha = 0.5, size = 0.8) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      color = "blue"
    ) +
    theme_minimal()
}

make_plot(10)
```



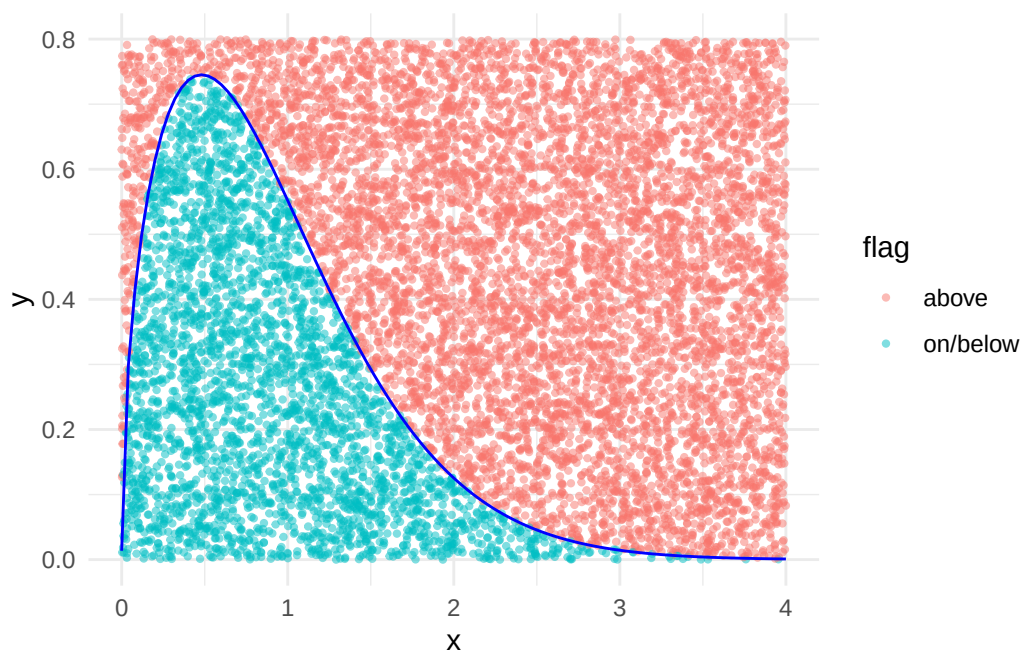

```
make_plot(100)
```



```r
make_plot(1000)
```

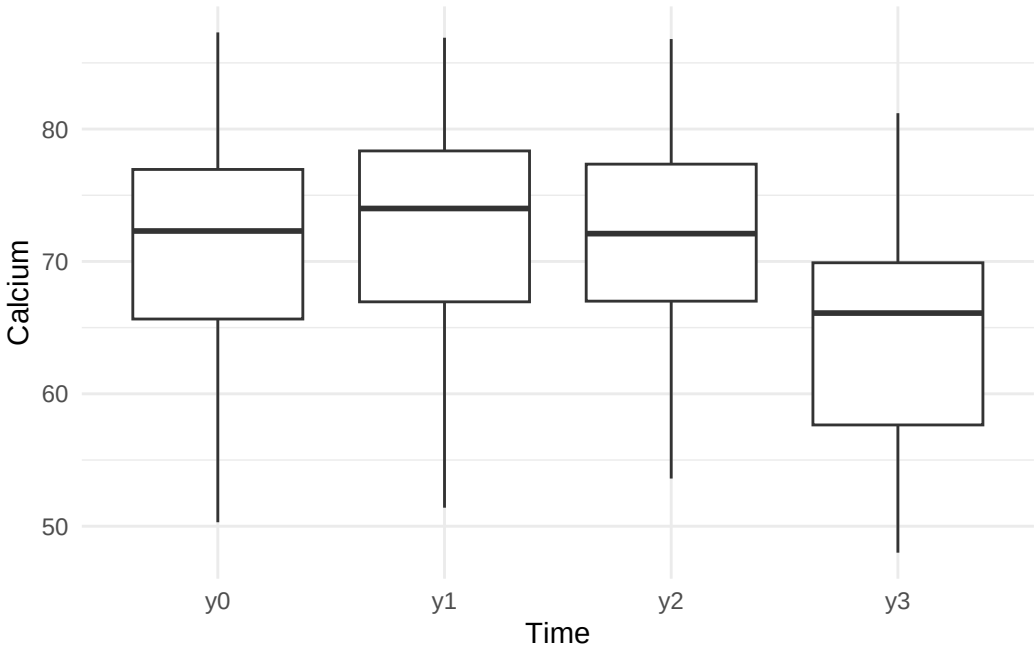



```r
make_plot(10000)
```



```r
## Calcium Data Analysis
df <- read.csv("/Users/kareenitakumar/Downloads/calcium.csv")

left_cols <- df[, 1:4]
right_cols <- df[, 5:8]

colnames(left_cols) <- c("y0","y1","y2","y3")
colnames(right_cols) <- c("y0","y1","y2","y3")

df_combined <- bind_rows(left_cols, right_cols)

df_combined$Subject_ID <- seq_len(nrow(df_combined))

df_long <- df_combined %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "Time",
    values_to = "Calcium"
  )

df_long$Time <- factor(df_long$Time, levels = c("y0","y1","y2","y3"))

summary_stats <- df_long %>%
  group_by(Time) %>%
  summarise(
    mean = mean(Calcium, na.rm = TRUE),
    sd   = sd(Calcium, na.rm = TRUE),
    min  = min(Calcium, na.rm = TRUE),
    max  = max(Calcium, na.rm = TRUE)
  )

summary_stats
```

```
# A tibble: 4 x 5
  Time   mean    sd   min   max
  <fct> <dbl> <dbl> <dbl> <dbl>
1 y0     70.8  9.72  50.3  87.3
2 y1     71.9  9.68  51.4  86.9
3 y2     71.8  8.78  53.6  86.8
4 y3     64.7  8.71  48    81.2
```

```r
ggplot(df_long, aes(x = Time, y = Calcium)) +
  geom_boxplot() +
  theme_minimal()
```

```
Warning: Removed 4 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```